A Comparative Study of Low-Code and Programmatic ETL Pipeline Architectures for Multi-Source Data Integration

1st Nadzla Andrita Intan Ghayatrie
Computer Science Department,
School of Computer Science,
Bina Nusantara University
Jakarta, Indonesia
nadzla.ghayatrie@binus.ac.id

2nd Joan Christina Bahagiono
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
joan.bahagiono@binus.ac.id

3rd Francis Jordan Hasnawi
Computer Science Department,
School of Computer Science,
Bina Nusantara University
Jakarta, Indonesia
francis.hasnawi@binus.ac.id

4th Fritz Gerald Norman
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
fritz.norman@binus.ac.id

5th William Chendra Kurniawan
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
william.kurniawan006@binus.ac.id

***Abstract*—The increasing need to integrate heterogeneous data from multiple sources has led to the development of various Extract, Transform, and Load (ETL) tools ranging from low-code to code-first approaches. The vast options available have led numerous studies to investigate and compare the functionality, performance, maintainability, and scalability of these tools, with the aim of achieving practical tool selection guidance. However, limited studies have explored the direct comparison between low code and code first ETL tools, especially with the same pipeline and across multiple datasets. This study aims to bridge the gap by replicating a public Pentaho pipeline to Python using equivalent transformation logic with the goal of creating a comparative analysis of the performance, complexity, auditability, and reproducibility in both approaches across three different datasets.**

Keywords—data warehouse, extract transform load, low code ETL tools, code first ETL tools, Pentaho, Python

I. INTRODUCTION

The need of heterogeneous data integration from multiple data sources for data analysis and insight gathering emerged in various sectors both in industry and research including education, healthcare, finance, retail, telecommunications, e-commerce, and tourism [1], [2]. Data heterogeneity refers to the diverse types and sources of the data. Heterogeneous data possess various challenges, including standardization, data quality, and scalability. These challenges are amplified in the field of big data as the data volume, velocity, variety, and veracity of data expands. In recent years, multiple studies have addressed these issues through proposed solutions related to data warehousing, distributed computing, cloud computing, and AI-aided data integration [3].

In the specific field of data warehousing, multiple extract, transform, and load (ETL) tools have been developed in the past, ranging from commercial to open-source tools [4]. These ETL tools aid in the process of (A) extracting data, which involves retrieving data from file systems, relational databases, or non-relational databases; (B) transforming the data, which involves data cleaning, data filtering, data transformation, and data versioning; and (C) loading the data into the intended target database. Cloud-based platforms such as Amazon Web Services, Google Cloud Platform, and Microsoft Azure also provided integrated ETL tools with different costs [5]. Highly customizable options for building ETL pipelines are also available such as through code-based approaches, for instance Python [6].

Various ETL pipeline development tools mainly fall under two main paradigms: low-code approaches which commonly use graphical components and connections to represent pipelines such as Pentaho, Talend, and Alteryx; and code-first approaches which provide direct control of transformation logic through source code such as Python, DBT, and Airflow. While each approach has its own advantages and disadvantages in performance and maintainability, tool selection is also influenced by other factors such as ease of use and tool learning curve [7]. In practice, developer preference and tool popularity may also influence the tool selection process. Previous research have compared the functionality, efficiency, and configurability of ETL tools [8], however, the comparison between low-code and code-first ETL tools is still underexplored, especially in an apple-to-apple comparison with equivalent functionalities in both paradigms and across multiple benchmark datasets.

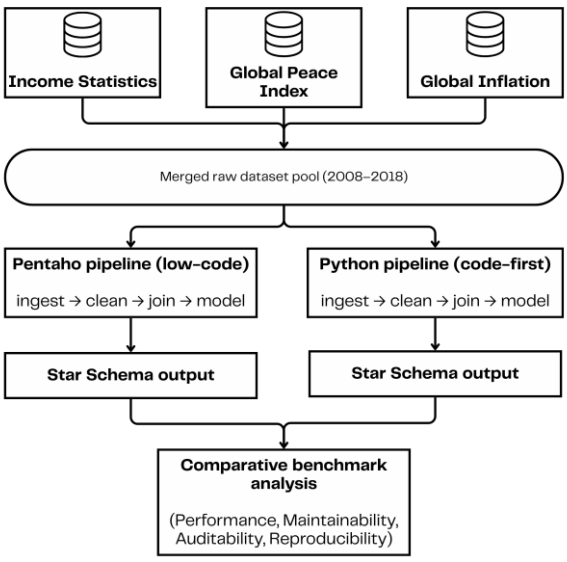


Fig. 1. Experiment Framework

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

This study contributes to bridging this gap by replicating a publicly available Pentaho ETL pipeline in Python/pandas using equivalent transformation logic with the goal of creating a comparative analysis of the performance, complexity, auditability, and reproducibility in both approaches. The comparison is experimented in three benchmark datasets. This study also aims to give practical recommendation in selecting between low-code and code-first approaches in small to medium scale heterogeneous data integration scenarios.

The remainder of this study is organized as follows. Section II reviews previous work on ETL tools comparison and star schema development for small-scale data warehousing. Section III describes the dataset and experimental methodology. Section IV discusses the results and limitations of the experiments. Section V presents the conclusion of the study and possible future works.

II. LITERARUTE REVIEW

A. Extract, Transform, and Load Tools Comparison

Recent literatures shows a diverse selection of ETL tools used to develop data integration pipelines. These ETL tools have been evaluated across various aspects including functionality, performance, scalability, and usability. Mahmud and Ikbal [9] reviews the evolution of early ETL tools to the emerging trends of modern ETL tools and analyzed the diverse ETL architectures present in enterprise and open-source tools. Kundavaram [10] compared the scalability aspect of traditional ETL tools against unified data platforms. Machireddy [11] discussed the importance of maximizing data quality management and techniques to optimize performance in enterprise ETL pipelines. Razali, Baharom, Abdullah, and Admodisastro [12] further emphasized the importance of data quality by implementing a quality scoring technique. Bósa and Heinzlreiter [13] explored different functionalities in low-code and code-first ETL tools while creating an automated data pipeline execution framework. However, only the functionality aspects of both approaches are discussed.

B. Data Warehouse Schema

In constructing a data warehouse, the selection of an appropriate schema is an important practical consideration as it can affect the efficiency of the data warehouse [14]. A star schema links dimension tables to fact tables, which is demonstrated by Wijaya, Wiratama, and Wijaya [15] in a property business data warehouse. Setiadi and Rahutomo [16] demonstrated that the implementation of a snowflake schema suits a larger scale data warehouse better compared to the star schema which is more appropriate for smaller scale data warehouses.

III. METHODOLOGY

This study aims to evaluate whether building a data pipeline that integrates multi-destination data sources is more effectively accomplished using low-code tools or a code-based (Python) approach [17]. Figure 1 presents the experimental workflow followed in this study.

This study uses three distinct data sources: the Global Income Statistics from the World Institute for Development Economics Research (WIDER), the Global Peace Index, and Global Inflation. Each dataset captures a country-level indicator, economic standing, peace level, and inflation rate respectively, which are then integrated over the 2008–2018 period to form a single global socio-economic analytical dataset. Two experiments were then carried out separately, using Pentaho and Python, to build the integration pipeline for this data. The outputs of the two pipelines were compared along four dimensions: performance, maintainability, auditability, and reproducibility.

A. Case Study Pipeline

The pipeline in this experiment integrates three heterogeneous public datasets (2008–2018): the Global Income Statistics (WIDER), covering the Gini Index and GDP per capita for 184 countries; the Global Peace Index, covering peace scores for 163 countries; and the Global Inflation Dataset, covering headline Consumer Price inflation for 212 countries.

TABLE I. TARGET SCHEMA ('FACT_ECONOMY_PEACE')

Column	Type	Description	Source Dataset
iso3c	String (PK, FK)	ISO 3-letter country code	Joint key across all sources
year	Integer	Observation year (2008–2018)	Joint key across all sources
peace_index	Numeric	Global Peace Index score	Global Peace Index
inflation_rate	Numeric	Headline Consumer Price Inflation (%)	Global Inflation Dataset
gini_index	Numeric	Gini inequality index	Global Income Statistics (WIDER)
gdp_val	Numeric	GDP per capita (PPP, constant 2011 USD)	Global Income Statistics (WIDER)

Since the two pipelines are being compared, a fair and equitable comparison requires both implementations to produce functionally equivalent output schemas, defined by a star schema consisting of one fact table and two dimension tables, as specified in Tables I, II, and III.

TABLE II. TARGET SCHEMA ('DIM_COUNTRY')

Column	Type	Description
iso3c	String (PK)	ISO 3-letter country code
country	String	Full country name

TABLE III. TARGET SCHEMA ('DIM_TIME')

Column	Type	Description
year	integer (PK)	Distinct observation year present in the fact table

B. Two Pipeline Implementations



Fig. 2. Experiment Framework

To ensure an apple-to-apple comparison, both pipelines in this study were built following an identical sequence of stage. As shown in Figure 2, this sequence consists of seven

consecutive stages: Data Collection, Data Ingestion, Data Storage, Data Transformation, Data Modelling, Data Quality & Validation, and ETL Orchestration.

Using an identical workflow across both implementations is methodologically important, as it ensures that any difference in benchmark results stems purely from differences in tooling paradigm rather than from the underlying pipeline logic.

1) Pipeline A (Low-Code): Four raw CSV files serve as input to this pipeline. Global_Dataset_of_Inflation.csv, Raw_WIDER_data.csv, WDI_GINI_data.csv, and peace_index.csv are each read through a separate CSV File Input step. The Data Transformation stage applies a Filter Rows step to restrict the observation period to 2008–2022, while discarding rows with missing values in gdp_val or gini_index. This is followed by a Unique Rows step that removes duplicate rows from the combined income sources prior to the join stage. Two sequential Merge Join steps then combine the result with the Peace Index, followed by Inflation, using Country and year as the join keys. The Data Modelling stage branches the final result into three outputs: Fact_Economy_Peace, Dim_Country, and Dim_Time. All 42 steps are orchestrated within a single Pentaho Job (.kjb), executed via Kitchen.

2) Pipeline B (Code-First): Pipeline B is implemented in Python 3 using the pandas library, realizing all seven stages of the reference workflow in Figure 1 through a single programmatic script (etl_pipeline.py). This approach represents a code-first ETL paradigm, in which transformation logic is expressed explicitly in code, written using libraries such as pandas so that data flows efficiently and consistently from source through transformation to the target system.

Data Collection & Ingestion. The three source datasets (WIDER, Global Peace Index, Global Inflation) are read through the functions ingest_income(), ingest_peace(), and ingest_inflation(), each handling a different source format (encoding, delimiter, and wide-to-long structure). Column type standardization is performed via pd.to_numeric() and astype().

Data Storage. The input (raw_dir) and output (out_dir) directories are separated as command-line parameters at script invocation, reflecting the separation between raw and processed data.

Data Transformation. The functions clean_income(), clean_peace(), and clean_inflation() handle null-value filtering and numeric format standardization. The join_all() function merges the three sources on the composite key (iso3c, year) using a pandas merge operation.

Data Modelling. The functions build_fact_table(), build_dim_country(), and build_dim_time() construct the output according to the target schema defined in Tables I, II, and III.

Data Quality & Validation. Deduplication is performed via drop_duplicates() on the key (iso3c, year) to ensure a single row per country-year combination.

ETL Orchestration. All stages are executed sequentially within the run_pipeline() function. A StageTimer class is included to record the execution duration of each stage, serving as the data collection instrument for the benchmark. This single-script design with explicitly named functions follows common practice in Python-based ETL pipeline development, where reproducible pipelines are typically built with minimal dependencies and organized as reusable, callable modules.

C. Comparative Benchmark Method

Each dimension in the benchmark analysis is measured using a different instrument, tailored to the characteristics of each pipeline. The four evaluation parameters were selected based on existing literature.

Performance and maintainability are drawn directly from the international standard ISO/IEC 25010 [18] which defines performance efficiency and maintainability as two of the nine primary characteristics of software product quality. This standard is a common reference across many tool evaluation and software comparison studies.

Auditability draws on the literature on data provenance and lineage in data pipelines. [19] emphasizes that the ability to trace the origin and transformation of data (provenance) is a prerequisite for a data pipeline to be auditable and for its results to be trustworthy. The Step Metrics capability in Pentaho and the StageTimer instrumentation in Python in this study are concrete implementations of that capability.

Reproducibility draws on [20], which formulates five pillars of computational reproducibility, one of which is repeatability, the ability of the same or a different researcher to reproduce a result using the same code and data, in a different environment, without manual modification.

IV. Results and Discussion

A. Functional Equivalence and Output Comparison

Before comparing performance and maintainability, this section first checks whether both pipelines process the same input under equivalent logic. This step matters because any divergence found in later metrics should come from the tooling paradigm itself, not from differences in how each pipeline was designed.

TABLE IV. Row count at key processing stages

Stage	Pentaho (rows)	Python (rows)	Match
Raw ingestion (4 CSV sources)	15,238	15,240	Close (2-row offset from header handling)
After range/null filtering (Filter Rows)	4,211	4,211	Identical
After deduplication (Unique Rows)	1,845	927	Diverges
After first join (income x peace)	2,106	Not applicable	n/a
After second join (+ inflation)	746	845	Diverges
Final fact table rows	747	845	Diverges
Final unique countries (Dim_Country)	131	148	Diverges
Final distinct years (Dim_Time)	12	11	Diverges

The row count after the filtering stage is identical between both pipelines (4,211 rows each) as shown in Table 4. This confirms that the year-range filter and the null-value exclusion on gini_index and gdp_val were replicated correctly in the Python implementation.

The two pipelines diverge starting at the deduplication stage. The Python implementation deduplicates on the composite business key (iso3c, year), so exactly one record survives per country-year combination, leaving 927 rows. Pentaho's Unique Rows step retains a much larger 1,845 rows. This gap suggests Pentaho's deduplication is evaluated across all columns rather than on the business key alone, so several records can survive for the same country-year whenever their non-key values differ even slightly, for example when Gini Index figures come from different underlying surveys. Because the join key is not actually unique going into the Merge Join steps, both merge operations multiply matching rows for any country-year that still has more than one surviving income record. This produces a many-to-many join effect in Pentaho, compared to the strict one-to-one join in the Python pipeline.

The final row and country counts (747/131 for Pentaho versus 845/148 for Python) follow directly from this upstream divergence rather than from any difference in the join key or join type used.

One discrepancy remains only partly explained. Dim_Time in Pentaho contains 12 distinct years, while the Python pipeline produces 11. Both the WIDER and WDI_GINI income sources only contain records for 2008 through 2018 inclusive, which is 11 years, matching the Python output. The extra year in Pentaho's Dim_Time could not be traced to a specific step within the scope of this study, since it is a downstream artifact of the multiplied Merge Join output rather than a value directly visible in Step Metrics. This is noted as an open limitation in Section V rather than resolved with an unverified explanation.

These findings are treated as auditability outcomes in Section 4.4, not as an error in either pipeline. Each pipeline behaves consistently with its own deduplication semantics; the semantics themselves are simply different, and the difference was not obvious from either pipeline's documentation alone.

B. Performance Comparison

Both pipelines were executed three times against the same four source files, on the same machine (Windows 11, Intel Core i9-13900H, 16GB RAM). Pentaho ran as Pentaho CE 11.0.0.0-237 via Kitchen. Python ran as a standalone script using pandas.

TABLE V. EXECUTION RUNTIME ACROSS 3 RUNS

Run	Pentaho (s)	Python (s)
1	2.244	0.203
2	1.718	0.160
3	1.741	0.153
Average	1.894	0.172

On identical hardware, Python completed the full pipeline roughly 11 times faster than Pentaho on average, as shown in Table 5. The gap is consistent across all three runs rather than driven by a single outlier, which suggests it reflects a genuine difference between the two execution models rather than measurement noise. A likely contributor is Pentaho's JVM startup and job orchestration overhead through Kitchen, which the single-script Python pipeline does not incur.

TABLE VI. PEAK MEMORY USAGE

Pipeline	Peak Memory
Pentaho	71.5 MB
Python	96.5 MB

Despite running faster, Python used about 35% more peak memory than Pentaho on the same machine, as shown in Table 6. This is consistent with how each tool processes data internally. Pandas loads full DataFrames into memory at each stage, while Kettle's engine processes rows in a streaming fashion, step by step, which keeps its memory footprint lower even when total runtime is higher. At the data volumes used in this case study (thousands of rows), the memory gap has no practical consequence, but it could matter more at a larger scale.

C. Maintainability and Complexity Comparison

TABLE VII. PEAK MEMORY USAGE

Metric	Pentaho	Python
Total steps/function	42 steps across all .ktr/.kjb files	11 named functions in one script
Number of pipeline artifact files	Multiple .ktr files plus 1 .kjb	1 .py file

Pentaho spreads its logic across more discrete visual steps and multiple files, coordinated by a single job file, as shown in Table 7. Python consolidates the same logic into a smaller set of named functions in one file. A lower step or function count does not automatically mean a pipeline is easier to maintain, but it does mean fewer discrete units a developer has to open and inspect one by one to understand the full logic. The Unique Rows ambiguity identified in Section 4.1 is a concrete case of this. Its exact deduplication behavior is not visible without opening the step's configuration dialog in Spoon, while the equivalent Python code, drop_duplicates(subset=["iso3c", "year"]), states its key directly at the point where it is called.

D. Auditability and Reproducibility

Row-level auditability. Pentaho's Step Metrics tab shows row counts in and out for every step, directly in the Spoon interface, without any extra setup. This was the exact mechanism used in Section 4.1 to trace where the two pipelines started to diverge. Python does not provide this out of the box. The per-stage row counts used in this paper only exist because a StageTimer class was deliberately added to the script during development. In other words, Pentaho's observability comes built in, while the same level of observability in a code-first pipeline has to be engineered on purpose.

Reproducibility and portability. The Pentaho transformation files contain a hardcoded absolute path. This means the .ktr file cannot run on a different machine, or be handed to a different collaborator, without manually editing the embedded path first. The Python script instead takes source and output locations as command-line arguments (--raw-dir, --out-dir), so the pipeline logic is not tied to any one machine's folder structure.

E. Discussion and Limitation

Neither tool wins on every dimension. Pentaho gives built-in row-level observability through Step Metrics and uses less

memory, but runs about 11 times slower, embeds machine-specific paths, and hid its deduplication behavior behind a step that took manual tracing to understand (Section 4.1). Python makes transformation logic explicit at the call site and runs faster and portably, but uses more memory and needs observability built in by hand. For integration tasks like this one, where join keys carry real ambiguity from multiple overlapping sources, that explicitness in code-first pipelines is the more direct safeguard against the kind of silent divergence found in Section 4.A.

A few limitations should be noted. First, the difference in Dim_Time row counts (12 in Pentaho versus 11 in Python) could not be traced to a specific step within this study's scope and is reported as an open question rather than a resolved one. Second, this comparison rests on a single case study and a single version of each tool (Pentaho CE 11.0.0.0-237, pandas 3.0.2), so the results may not generalize to other pipelines, tool versions, or data volumes. Third, the maintainability comparison in Section 4.C counts Pentaho steps against Python functions, two units that are not strictly equivalent, and should be read as an approximate indicator rather than a precise measurement.

V. Conclusion

This study benchmarked a low-code pipeline (Pentaho) against a code-first pipeline (Python/pandas) built to replicate the same case study, an integration of four heterogeneous socio-economic datasets into a star schema. Both pipelines produced functionally comparable output, but diverged in row counts due to a difference in how each handles deduplication, a divergence that was only traceable through row-level Step Metrics rather than visible from either pipeline's surface design. On the same machine, Python ran roughly 11 times faster and used more memory, while Pentaho offered built-in observability that Python only matched through manual instrumentation, and Pentaho's transformation files carried a hardcoded local path that Python's did not.

Overall, no single tool is superior across every dimension measured. Low-code and code-first pipelines involve real trade-offs between speed, resource use, built-in observability, and the explicitness of transformation logic, and the right choice depends on which of these matters most for a given task. Future work could extend this comparison to larger data volumes, additional tools such as dbt or Airflow, and a resolution of the unexplained Dim_Time discrepancy noted in Section 4.E.

Acknowledgment *(Heading 5)*

The authors would like to thank Bina Nusantara University for its support throughout this research.

Funding. This research received no grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of Interests. The authors declare no conflict of interest.

Data Availability. The datasets used in this study, Global Income Statistics (WIDER), Global Peace Index, and Global Inflation, are publicly available and can be accessed via the Kaggle platform at https://www.kaggle.com/.

Author Contribution. *Nadzla Andrita Intan Ghayatrie:* Conceptualization, Writing, Methodology, Validation, Experiment, Funding Acquisition. *Joan Christiana Bahagiono:* Conceptualization, Writing, Validation. *Francis Jordan Hasnawi:* Conceptualization, Methodology, Experiment. *Fritz Gerald Norman:* Conceptualization, Methodology, Experiment. *William Chendra Kurniawan:* Conceptualization, Methodology, Experiment.

References

[1] W. Z. Alma'Aitah, A. Quraan, F. N. Al-Aswadi, R. S. Alkhawaldeh, M. Alazab, and A. Awajan, "Integration Approaches for Heterogeneous Big Data: A Survey," *Cybernetics and Information Technologies*, vol. 24, no. 1, pp. 3–20, Mar. 2024, doi: 10.2478/cait-2024-0001.

[2] I. M. Putrama and P. Martinek, "Heterogeneous data integration: Challenges and opportunities," Oct. 01, 2024, *Elsevier Inc.* doi: 10.1016/j.dib.2024.110853.

[3] F. Z. Rozony, M. N. A. Aktar, M. Ashrafuzzaman, and A. Islam, "A Systematic Review Of Big Data Integration Challenges And Solutions For Heterogeneous Data Sources," *Academic Journal On Business Administration, Innovation & Sustainability*, vol. 4, no. 4, pp. 1–18, Oct. 2024, doi: 10.69593/ajbais.v4i04.111.

[4] M. Szilveszter, G. Molnar, and E. Nagy, "Comparison of Commercial and Open Source ETL Tools," in *ICCC 2024 - IEEE 11th International Conference on Computational Cybernetics and Cyber-Medical Systems, Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 37–42. doi: 10.1109/ICCC62278.2024.10582972.

[5] P. Borra, "Comparative Review: Top Cloud Service Providers Etl Tools-Aws Vs. Azure Vs. Gcp," *International Journal of Computer Engineering and Technology (IJCET)*, vol. 15, no. 3, pp. 203–208, 2024, doi: 10.17605/OSF.IO/X7WCT.

[6] B. K. Pandey and E. Ro Schoof, "Building ETL Pipelines with Python: Create and deploy enterprise-ready ETL pipelines by employing modern methods.," in *Building ETL Pipelines with Python*, 2023, pp. 1–246.

[7] A. Qaiser, M. U. Farooq, S. Muhammad, N. Mustafa, and N. Abrar, "Comparative Analysis of ETL Tools in Big Data Analytics," *Pakistan Journal of Engineering and Technology*, vol. 6, pp. 7–12, 2023.

[8] S. R. Cheruku, O. Goel, and S. Jain, "A Comparative Study of ETL Tools: DataStage vs. Talend," *Journal of Quantum Science and Technology*, vol. 1, no. 1, pp. 80–90, Mar. 2024, doi: 10.36676/jqst.v1.i1.11.

[9] D. Mahmud and Md. Z. Ikbal, "The Role Of Etl (Extract-Transform-Load) Pipelines In Scalable Business Intelligence: A Comparative Study Of Data Integration Tools," *ASRC Procedia: Global Perspectives in Science and Scholarship*, vol. 02, no. 01, pp. 89–121, Jan. 2022, doi: 10.63125/1spa6877.

[10] V. N. K. Kundavaram, "Scaling Data Engineering With Advanced Data Management Architecture: A Comparative Analysis Of Traditional Etl Tools Against The Latest Unified Platform," *International Journal for Advanced Research in Science and Technology*, vol. 13, p. 7, 2023.

[11] J. Reddy Machireddy, "Data Quality Management and Performance Optimization for Enterprise-Scale ETL Pipelines in Modern Analytical Ecosystems," *Journal of Data Science, Predictive Analytics, and Big Data Applications*, vol. 8, no. 7, pp. 1–26, 2023.

[12] N. F. A. Razali, S. Baharom, S. Abdullah, and N. I. Admodisastro, "Improving Data Reliability Assessment in ETL Processes through Quality Scoring Technique in Data Analytics," *International Journal On Informatics Visualization*, vol. 8, no. 4, 2024, [Online]. Available: www.joiv.org/index.php/joiv

[13] K. Bósa and P. Heinzlreiter, "Automated Execution of Data Pipelines based on Configuration Files.," *Open Research Europe*, vol. 5, 2025, doi: 10.12688/openreseurope.21019.1.

[14] A. Nambiar and D. Mundra, "An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management," *Big Data and Cognitive Computing*, vol. 6, no. 4, Dec. 2022, doi: 10.3390/bdcc6040132.

[15] W. Wijaya, J. Wiratama, and S. F. Wijaya, "Implementation of Data Warehouse and Star Schema for Optimizing Property Business Decision Making," *G-Tech: Jurnal Teknologi Terapan*, vol. 8, no. 2, pp. 1242–1250, Apr. 2024, doi: 10.33379/gtech.v8i2.4091.

[16] T. F. Setiadi and R. Rahutomo, "A Data Warehouse Schema for Monitoring Regional COVID-19 Case Registration," in *Proceedings of 2022 International Conference on Information Management and Technology, ICIMTech 2022*, Institute of

Electrical and Electronics Engineers Inc., 2022, pp. 144–148. doi: 10.1109/ICIMTech55957.2022.9915222.

[17] D. Pinho, A. Aguiar, and V. Amaral, "What about the usability in low-code platforms? A systematic literature review," *J. Comput. Lang.*, vol. 74, p. 101185, Jan. 2023, doi: 10.1016/j.cola.2022.101185.

[18] International Organization for Standardization, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," Nov. 2023.

[19] L. Rupprecht, J. C. Davis, C. Arnold, Y. Gur, and D. Bhagwat, "Improving reproducibility of data science pipelines through transparent provenance capture," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3354–3368, Aug. 2020, doi: 10.14778/3415478.3415556.

[20] M. Ziemann, P. Poulain, and A. Bora, "The five pillars of computational reproducibility: bioinformatics and beyond," *Brief. Bioinform.*, vol. 24, no. 6, Sep. 2023, doi: 10.1093/bib/bbad375.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove template text from your paper may result in your paper not being published.